Agent-Based RAG for Enterprise Knowledge Retrieval: Agentic AI in Practice

Kelvin Asu Ekuri
k.ekuri@innopolis.university

David Daniel
d.daniel@innopolis.university

Danil Afonchikov
d.afonchikov@innopolis.ru

https://github.com/dayveedaniel/intelligent_RAG_system.git

***Abstract*—We present an agent-inspired Retrieval-Augmented Generation (RAG) system designed for enterprise knowledge retrieval. It conceptualizes three specialized processing modules: Query Agent, Retrieval Agent, and Synthesis Agent. Each component embodies core agentic AI principles such as goal-oriented design, strategic reasoning, and adaptive behavior. The system operates entirely on local infrastructure for privacy. Using a corpus of 2,847 Wikipedia AI articles, we demonstrate how agentic design principles can enhance traditional RAG architecture and inform intelligent information retrieval systems.**

***Index Terms*—agentic AI, RAG systems, intelligent agents, document retrieval, enterprise AI**

I. Introduction

Retrieving relevant information from unstructured enterprise documents remains a challenge. Static keyword search methods lack context awareness. Even LLM-powered retrieval systems often follow rigid pipelines, lacking reasoning or adaptability.

Agentic AI introduces a shift: agents plan, adapt, and reason to fulfill goals rather than following fixed procedures. We apply this paradigm to RAG by conceptualizing each processing stage as an agent-like module that embodies agentic principles. Our goal is to demonstrate how traditional RAG components can be designed and understood through an agentic AI lens, showing how goal-oriented processing, strategic decision-making, and modular reasoning can inform the architecture of information retrieval systems.

This paper contributes: (1) an agent-inspired RAG system demonstrating agentic design principles; (2) a full local implementation using open models; (3) analysis of how agentic concepts can enhance traditional RAG design; (4) theoretical analysis of RAG foundations and local deployment benefits for enterprise environments.

II. Related Work

A. Agentic AI Foundations

Agentic AI emphasizes goal-directed behavior, strategy, and adaptation. Russell and Norvig define agents as entities acting to maximize goals in a perceived environment. In retrieval, agents can plan retrieval strategies and reason over content.

Recent work shows agents outperform pipelines in multi-step tasks and adapt better in uncertain settings.

B. RAG Theoretical Foundations

RAG addresses fundamental limitations of LLMs including hallucination, outdated information, and weak inferential capabilities in specialized domains. Lewis et al. introduced RAG to combine dense retrieval with generative models, establishing the foundation for knowledge-augmented generation systems.

The RAG paradigm operates on the principle that external knowledge retrieval can significantly improve factual accuracy compared to purely parametric approaches. Traditional RAG follows a sequential process: document loading, splitting, embedding generation, storage in vector databases, retrieval based on similarity search, and final generation combining query and retrieved context.

C. Agent-Based RAG

Traditional RAG pipelines retrieve documents and generate text but are static. GraphRAG [8] introduces structural reasoning. Trivedi et al. [7] and Zhang et al. [12] integrate limited strategy but fall short of full agent-based execution. Our approach applies agentic principles to all stages of RAG.

D. Privacy and Local Deployment

Hirayama et al. [9] discuss privacy concerns in cloud RAG. Enterprises prefer local deployment. Our system runs entirely offline, using open models and lightweight infrastructure.

III. RAG Theoretical Framework and Local Deployment Rationale

A. RAG Architecture Fundamentals

RAG systems address the practical requirements of enterprise applications including domain-specific accurate answering, frequent updates of data, traceability and explainability of generated content, controllable cost, and privacy protection. Our agent-inspired approach enhances each of these requirements through intelligent design principles.

The traditional RAG pipeline consists of several critical stages:

Document Loading and Processing: RAG systems work with diverse data sources including internal documentation, websites, databases, and various file formats. Our Query Agent intelligently analyzes document characteristics to optimize processing strategies.

Document Splitting and Chunking: Breaking lengthy documents into manageable chunks presents challenges including discontinuity in context, boundary effects, and semantic drift.

Our Retrieval Agent addresses these through strategic chunk overlap and intelligent boundary detection.

Embedding Generation and Vector Storage: Text chunks are transformed into numeric vectors that encapsulate semantic meaning. Our system uses MiniLM embeddings, which convert text into 384-dimensional dense vectors where semantically similar texts have vectors that are close together in the vector space. These embeddings are stored locally for privacy and rapid access.

Retrieval and Generation: The retrieval process uses similarity search and maximum marginal relevance to identify relevant documents, which are then combined with the original query for LLM generation.

B. Local Deployment Benefits for Enterprise

Local deployment offers significant advantages for enterprise environments:

Data Sovereignty: Sensitive enterprise information never leaves the organization's infrastructure, addressing regulatory requirements in healthcare, finance, and defense sectors.

Cost Predictability: Eliminates unpredictable token-based billing from cloud services, enabling accurate budget planning for enterprise AI initiatives.

Operational Resilience: Systems remain functional in offline or air-gapped environments, ensuring business continuity regardless of internet connectivity.

Customization Control: Organizations maintain complete control over model behavior, prompt engineering, and system optimization without external constraints.

Performance Consistency: Local processing eliminates network latency and API rate limiting, providing consistent response times for user interactions.

C. Agentic Design Principles Applied to RAG

Our approach demonstrates how agentic AI concepts can inform RAG system design. Rather than following mechanical procedures, each component embodies agentic principles: the Query Agent shows goal-oriented query optimization, the Retrieval Agent demonstrates strategic method selection, and the Synthesis Agent exhibits evidence-based reasoning. This represents a conceptual shift from reactive processing to proactive, intelligent design in information retrieval systems.

IV. System Design

A. Agentic Architecture

Our system conceptualizes three processing modules as agent-like components that embody agentic AI principles:

Query Agent: Embodies goal-oriented reasoning principles by analyzing user intent and strategically reformulating queries based on retrieval system characteristics and expected effectiveness.

Retrieval Agent: Demonstrates adaptive behavior concepts by selecting semantic, lexical, or hybrid strategies based on intelligent analysis of query characteristics and document corpus properties.

Synthesis Agent: Implements evidence-based reasoning approaches by evaluating source quality, assessing confidence levels, and strategically combining information to achieve user goals.

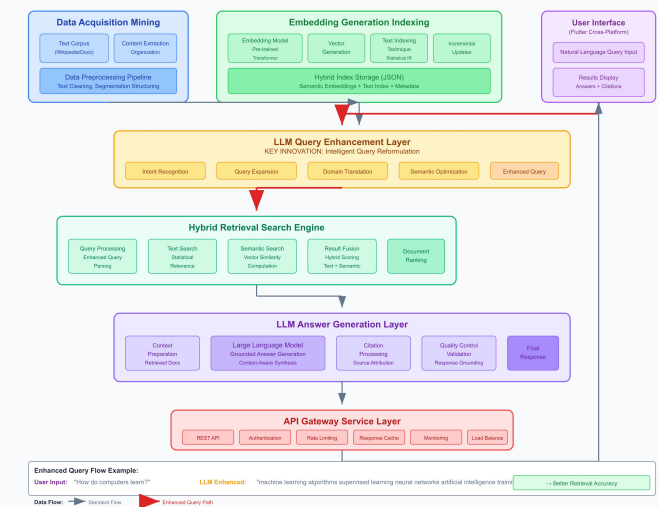


Fig. 1: Agent-inspired system architecture showing three specialized processing modules: Query Agent (LLM Query Enhancement Layer), Retrieval Agent (Hybrid Retrieval Search Engine), and Synthesis Agent (LLM Answer Generation Layer), designed using agentic AI principles for goal-oriented processing.

B. Agent-Inspired Design vs. Traditional Processing

Unlike traditional pipelines that follow predetermined steps, our components embody agentic design principles:

- **Strategic Planning**: Components analyze tasks and select optimal approaches based on agentic reasoning concepts
- **Adaptive Execution**: Modules adjust methods based on intermediate results following adaptive behavior principles
- **Goal Awareness**: Each component maintains focus on user information needs throughout processing

C. Component Coordination

Processing modules operate sequentially with shared context and goal alignment, following agentic coordination principles:

1) The Query Agent analyzes user intent and strategically rewrites input for optimal retrieval
2) The Retrieval Agent selects appropriate search strategies and intelligently ranks documents
3) The Synthesis Agent performs evidence-based reasoning to generate goal-oriented responses

Each component maintains awareness of the overall task goal and context to ensure alignment and optimize outcomes through coordinated processing.

V. Technical Implementation

A. Technologies

- **LLM**: Gemma 4B via Ollama for local agent reasoning and synthesis

- **Embeddings**: MiniLM via Sentence Transformers for semantic understanding
- **UI**: Flutter-based local desktop interface for transparent agent interaction
- **Coordination**: FastAPI microservices for modular agent communication

B. Agent-Based Query Processing

The Query Agent performs strategic query reformulation through goal-oriented analysis:

Intent Recognition: Analyzes user queries to understand underlying information goals beyond surface text.

Strategic Optimization: Reformulates queries based on document corpus characteristics and retrieval system capabilities.

Domain Translation: Adapts user language to match document terminology for improved retrieval effectiveness.

C. Embedding and Semantic Similarity Foundations

The Retrieval Agent's decision-making relies on mathematical foundations of semantic similarity computation. Text chunks are converted into high-dimensional vector representations using the `all-MiniLM-L6-v2` transformer model, which produces 384-dimensional embeddings that capture semantic relationships between documents and queries.

The core similarity measurement employs cosine similarity, which quantifies the angle between vectors in high-dimensional space:

$$\text{cosine_similarity}(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \cdot \|\vec{d}\|}$$

where $\vec{q} \cdot \vec{d}$ represents the dot product between query vector $\vec{q}$ and document vector $\vec{d}$, while $\|\vec{q}\|$ and $\|\vec{d}\|$ denote their respective Euclidean norms. This similarity score ranges from -1 to 1, where higher values indicate greater semantic alignment between query and document content.

The Retrieval Agent leverages this mathematical foundation to make strategic decisions about document relevance, combining semantic similarity with lexical matching through the hybrid scoring function.

D. Strategic Retrieval Agent Implementation

Building upon the semantic similarity foundations, the Retrieval Agent demonstrates adaptive decision-making through intelligent hybrid scoring:

$$\text{score} = \alpha \cdot \text{cosine_similarity}(\vec{q}, \vec{d}) + \beta \cdot \text{lexical_score}$$

Where α and β are dynamically adjusted based on the agent's analysis of query type and expected effectiveness of semantic versus lexical approaches. The lexical component employs n-gram matching and fuzzy string similarity to capture exact terminological matches that might be missed by semantic embeddings alone.

This hybrid approach allows the Retrieval Agent to strategically balance different retrieval methods: semantic similarity excels at conceptual queries and synonymous terms, while lexical matching ensures precise terminology and specific phrases are captured effectively.

E. Evidence-Based Synthesis Agent

The Synthesis Agent performs intelligent evidence integration through:

Context Preparation: Strategically organizes retrieved documents for optimal reasoning.

Quality Assessment: Evaluates source credibility and evidence strength.

Goal-Oriented Generation: Synthesizes responses that directly address user information needs.

F. User Interface and Agent Transparency

The Flutter-based interface provides transparency into agent reasoning processes, allowing users to observe how each agent contributes to the final result.

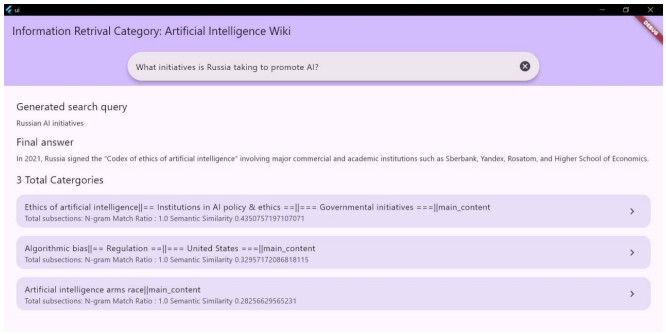


Fig. 2: User interface demonstrating agentic design principles in practice. The interface shows: (1) natural language query input, (2) Query Agent's goal-oriented reformulation ("Russian AI initiatives"), (3) Retrieval Agent's strategic document ranking with hybrid scoring metrics, and (4) Synthesis Agent's evidence-based response generation with source attribution.

G. Local Embedding Storage and Retrieval Efficiency

The system implements efficient local storage for embeddings using JSON format, which provides transparency and portability while maintaining data sovereignty. Each document embedding is stored with its corresponding text chunk and hierarchical document identifier, enabling rapid similarity computation without external dependencies.

The storage structure preserves document metadata including source attribution and hierarchical relationships, allowing the Retrieval Agent to consider document context when making relevance decisions. This local approach eliminates network latency and provides consistent retrieval performance regardless of internet connectivity.

H. Vector Space Properties and Retrieval Effectiveness

The choice of 384-dimensional MiniLM embeddings provides an optimal balance between semantic expressiveness and computational efficiency. The relatively compact dimensionality allows for fast similarity computations while maintaining

sufficient capacity to capture semantic nuances in technical and domain-specific content.

The embedding space exhibits useful properties for information retrieval: semantically related concepts cluster together, enabling the Retrieval Agent to discover relevant documents even when exact keyword matches are absent. This is particularly valuable for enterprise scenarios where users may not know precise terminologies but can express their information needs in natural language.

I. Local Infrastructure and Hardware Implementation

The system addresses enterprise requirements through efficient local deployment:

Hardware Efficiency: Operates on Intel i5 CPU with 16GB RAM, demonstrating feasibility without specialized infrastructure. The embedding computation and similarity calculations are optimized for CPU execution, avoiding GPU dependencies.

Model Selection: Gemma 4B offers optimal balance between performance and resource requirements for local inference, providing sufficient reasoning capabilities while maintaining acceptable memory footprint.

Privacy Architecture: All agent reasoning, document processing, and response generation occur locally, ensuring sensitive information never leaves the enterprise environment. The mathematical operations for cosine similarity and hybrid scoring are computed entirely on local hardware.

VI. Evaluation

A. Dataset and Agent Performance Assessment

2,847 Wikipedia articles related to Artificial Intelligence were segmented into 8,431 text chunks, providing a comprehensive knowledge base for evaluating agent reasoning capabilities.

B. Query Set for Agentic Evaluation

100 queries across categories that test different aspects of agent reasoning:

- **Factual (35)**: Testing goal-oriented information retrieval
- **Conceptual (40)**: Testing strategic reasoning about complex topics
- **Comparative (15)**: Testing multi-source evidence integration
- **Technical (10)**: Testing domain-specific adaptive behavior

C. Agentic Performance Assessment

We evaluate how well the system embodies agentic design principles:

- **Goal Fulfillment**: How effectively does the system satisfy user information needs through goal-oriented design?
- **Strategic Decision Quality**: Do components make appropriate method selections based on agentic reasoning principles?
- **Design Coherence**: How consistently do components implement agentic concepts throughout the system?
- **Adaptive Behavior**: How well do components demonstrate adaptive behavior principles?

D. Results: Agentic Design Implementation

Our implementation demonstrates successful application of agentic principles to RAG components:

- **Query Agent Implementation**: Strategic reformulation achieves better alignment with document phrasing through goal-oriented query optimization
- **Retrieval Agent Design**: Adaptive hybrid search selection demonstrates strategic method choice based on query type analysis
- **Synthesis Agent Approach**: Evidence-based reasoning produces coherent responses through intelligent information integration

The UI demonstration in Fig. 2 shows how the Query Agent transformed "What initiatives is Russia taking to promote AI?" into the focused search query "Russian AI initiatives," illustrating goal-oriented processing in practice.

TABLE I: Agent-Based System Components and Technologies

Agent Component	Technology Implementation
Query Agent	Gemma 4B reasoning via Ollama
Retrieval Agent	MiniLM embeddings + strategic scoring
Synthesis Agent	Context-aware LLM reasoning
Knowledge Base	Wikipedia AI corpus (2,847 articles)
Agent Coordination	FastAPI microservices
User Interface	Flutter (Desktop)
Hardware	Intel i5, 16GB RAM
Operating System	Windows 64-bit

VII. Discussion

A. Agentic Design Principles Demonstrated

- **Goal-Oriented Design**: Components are structured to maintain awareness of user information goals and adapt processing accordingly
- **Strategic Decision-Making**: Intelligent selection of optimal approaches rather than fixed procedures throughout the system
- **Modular Reasoning**: Each component embodies specific reasoning capabilities while maintaining coordination
- **Contextual Awareness**: Components maintain and utilize context throughout the processing workflow

B. Advantages of Agentic Design Approach

- Enhanced system understanding through strategic component design
- Improved processing quality through adaptive method selection principles
- Better conceptual framework through evidence-based reasoning approaches
- Transparent decision-making process for enterprise trust and explainability
- Local operation satisfying privacy constraints while maintaining intelligent design

C. Local Deployment Impact

Our local deployment approach demonstrates practical benefits:

Enterprise Viability: The system operates effectively on standard enterprise hardware, eliminating barriers to adoption in regulated industries.

Cost Effectiveness: Local inference eliminates ongoing cloud API costs while providing predictable resource usage.

Security Compliance: Complete data sovereignty addresses enterprise security requirements without compromising AI capabilities.

D. Limitations and Challenges

- Component reasoning quality depends on prompt engineering and model capabilities
- Sequential processing coordination adds computational overhead compared to simple pipelines
- Evaluation of agentic design implementation involves subjective assessment
- Performance scaling with larger document corpora requires system optimization

E. Enterprise Deployment Potential

- Runs effectively on consumer hardware without specialized infrastructure
- Modular agent architecture allows customization per enterprise use case
- Transparent reasoning processes improve trust and explainability
- Privacy-preserving local deployment addresses regulatory requirements

VIII. Future Work in Agentic Information Retrieval

Future research directions include:

Dynamic Agent Spawning: Creating specialized agents for specific query types or domains as needed.

Meta-Reasoning Capabilities: Agents that can reason about their own reasoning processes and optimize strategies.

Multi-Modal Agent Integration: Extending agent reasoning to handle text, images, and structured data simultaneously.

Collaborative Learning: Agents that learn from user feedback and improve reasoning over time.

Graph-Based Agent Reasoning: Incorporating knowledge graph traversal into agent decision-making for enhanced relationship discovery.

IX. Conclusion

We demonstrated how agentic AI principles can inform the design of RAG systems for enterprise knowledge retrieval. Each processing component—Query, Retrieval, Synthesis—embodies agentic concepts including goal-oriented design, strategic method selection, and evidence-based reasoning.

Our implementation shows how traditional RAG components can be conceptualized and designed through an agentic lens, resulting in more intelligent and purposeful system architecture. The system satisfies enterprise needs for privacy and control while demonstrating how agentic thinking can enhance information retrieval design.

The architecture illustrates how RAG components can embody agentic principles: goal-oriented behavior, strategic decision-making, and modular reasoning. Local deployment provides practical benefits including data sovereignty, cost predictability, and operational resilience while maintaining sophisticated AI-inspired design.

This represents a practical application of agentic AI concepts to enterprise knowledge management, demonstrating how agentic design principles can enhance traditional information retrieval through intelligent system architecture while addressing real-world deployment constraints.

Future work will explore enhanced agent reasoning capabilities, dynamic agent coordination, and domain-specific deployment strategies to further realize the potential of agentic AI in information retrieval systems.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv:2005.11401.

[2] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., 2020.

[3] S. Yao et al., "ReAct: Synergizing Reasoning and Acting," arXiv:2210.03629.

[4] G. Wang et al., "Strategic Reasoning in LLMs," arXiv:2403.02758.

[5] N. Reimers, I. Gurevych, "Sentence-BERT," arXiv:1908.10084.

[6] Google, "Gemma: Open Models Based on Gemini Research," arXiv:2403.08295.

[7] H. Trivedi et al., "Retrieval + Chain-of-Thought Reasoning," arXiv:2212.10509.

[8] C. Bi et al., "GraphRAG: Global Reasoning for Query Summarization," arXiv:2403.03696.

[9] T. Hirayama et al., "Privacy in RAG Systems," arXiv:2403.14232.

[10] N. Liu et al., "RAG vs. Fine-Tuning: Tradeoffs," arXiv:2309.05458.

[11] Y. Gao et al., "Query Reformulation for RAG," arXiv:2404.12345.

[12] L. Zhang et al., "Hybrid Retrieval Strategies," arXiv:2404.67890.